Rank–Nullity Theorem

An End-to-End Meta-Math Walkthrough

Dibakar Sigdel[1]

[1] *Mindverse Computing LLC, Lynnwood, WA 98087, USA*

(Dated: October 2026)

Abstract

A complete authored walkthrough of rank–nullity theorem, with explicit mathematical contracts, local verification gates, and reusable exports. The report presents the target theorem, declared imports, a dependency graph, five-faculty node contracts, a continuous proof, a worked example, and a hypothesis-repair audit. It is an authored mathematical exposition, not a deployed-agent transcript or a formal proof-assistant certificate.

LA-01 / Linear Algebra / Report v1

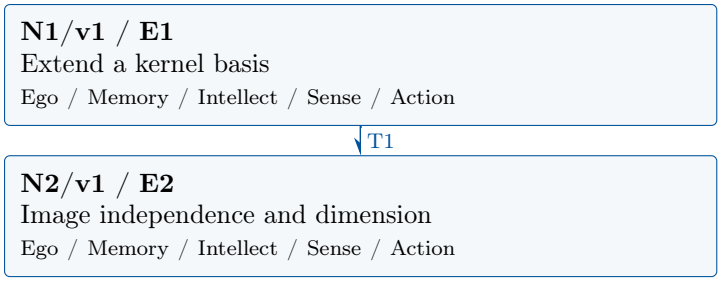
I. TARGET AND IMPORT CONTRACT

Theorem I.1. *For a linear map $T : V \to W$ with finite-dimensional V, prove* $\dim V = \dim \ker T + \dim \operatorname{im} T$.

A. Foundations and permitted imports

All vector spaces are over one field F. V is finite-dimensional; W need not be. Import basis extension in a finite-dimensional vector space, and the definitions of kernel, image, linear independence, and span. The zero-dimensional cases use an empty basis.

II. NODE INTERACTION WORKFLOW



Each box is one mathematical agent containing all five faculties. Blue arrows carry the versioned exports listed below. The layout gives a reading order; only drawn arrows assert dependencies. Diagnostic repair is separate from the proof DAG.

A. Versioned evidence-transfer ledger

T1: N1/E1/v1 to N2. The images $Tv_1, \ldots, Tv_r$ span the image.

Receiving activity. N2: Image independence and dimension

Compatibility. Use the same mathematical objects and hypotheses as the source export. Receiving contract: Extended domain basis; prior exports E1/v1

III. AGENT ROLES AND FIVE-FACULTY ACTIVITIES

A. N1/v1: Extend a kernel basis

Object and role. Complete the basis in the domain, not the codomain.

Inputs. T linear; $u_1, \ldots, u_k$ basis of kerT; foundational imports stated in §1

E — Ego

Own E1/v1: The images $Tv_1, \ldots, Tv_r$ span the image.

M — Memory

Retain T linear; $u_1, \ldots, u_k$ basis of kerT; foundational imports stated in §1; keep their assumptions and versions attached.

I — Intellect

Complete the basis in the domain, not the codomain.

S — Sense

The added basis vectors must complement the whole kernel.

A — Action

Carry out the numbered deductions and publish E1/v1 together with its hypotheses and proof location.

Numbered mathematical activities

N1.1 Choose a basis $u_1, \ldots, u_k$ of ker T and extend it to $u_1, \ldots, u_k, v_1, \ldots, v_r$ of V.

N1.2 Thus $\dim V = k + r$.

N1.3 For any $x \in V$ write $x = \sum a_i \; u_i + \sum b_j \; v_j$.

N1.4 ApplyingT yields $Tx = \sum b_j Tv_j$ because every u_i lies in the kernel.

N1.5 Hence the vectors Tv_j span im T.

Export E1/v1. The images $Tv_1, \ldots, Tv_r$ span the image.

Acceptance gate. The added basis vectors must complement the whole kernel. Publish only after checking every displayed deduction.

Targeted repair. If the check fails, revise N1: The added basis vectors must complement the whole kernel. Recheck dependent nodes N2.

B. N2/v1: Image independence and dimension

Object and role. Pull a zero combination back to the kernel.

Inputs. Extended domain basis; prior exports E1/v1

E — Ego

Own E2/v1: Rank–nullity with no finiteness requirement on W.

M — Memory

Retain Extended domain basis; prior exports E1/v1; keep their assumptions and versions attached.

I — Intellect

Pull a zero combination back to the kernel.

S — Sense

A spanning list alone cannot be counted as a basis.

A — Action

Carry out the numbered deductions and publish E2/v1 together with its hypotheses and proof location.

Numbered mathematical activities

N2.1 If $\sum b_j T v_j = 0$ then $\sum b_j v_j$ lies inkerT, so equals $\sum a_i\ u_i$.

N2.2 Independence of the extended basis forces all b_j and a_i zero.

N2.3 Thus Tv_j are independent and form a basis ofimT.

N2.4 Therefore $\dim \operatorname{im} T = r$ and $\dim V = k + r = \dim \ker T + \dim \operatorname{im} T$.

N2.5 If $r = 0$ this is the empty image basis; if $k = 0$ it states injective maps preserve domain dimension.

Export E2/v1. Rank–nullity with no finiteness requirement on W.

Acceptance gate. A spanning list alone cannot be counted as a basis. Publish only after checking every displayed deduction.

Targeted repair. If the check fails, revise N2: A spanning list alone cannot be counted as a basis. Withhold this terminal result until its gate passes.

IV. CONTINUOUS MATHEMATICAL PROOF

Read these steps in order as one proof; the diagram is optional. Numbered activities identify the specific constructions and deductions owned by each node.

A. §4.N1 — Extend a kernel basis

1. Choose a basis $u_1, \ldots, u_k$ of ker T and extend it to $u_1, \ldots, u_k, v_1, \ldots, v_r$ of V.
2. Thus $\dim V = k + r$.
3. For any $x \in V$ write $x = \sum a_i \; u_i + \sum b_j \; v_j$.
4. ApplyingT yields $Tx = \sum b_j T v_j$ because every u_i lies in the kernel.
5. Hence the vectors Tv_j span im T.

B. §4.N2 — Image independence and dimension

1. If $\sum b_j T v_j = 0$ then $\sum b_j v_j$ lies inkerT, so equals $\sum a_i \; u_i$.
2. Independence of the extended basis forces all b_j and a_i zero.
3. Thus Tv_j are independent and form a basis ofimT.
4. Therefore $\dim \operatorname{im} T = r$ and $\dim V = k + r = \dim \ker T + \dim \operatorname{im} T$.
5. If $r = 0$ this is the empty image basis; if $k = 0$ it states injective maps preserve domain dimension.

□

V. WORKED ORCHESTRATION

For $T : R^3 \to R^2$, $T(x, y, z) = (x + y, z)$, the kernel has $basis(1, -1, 0)$. Extend by (0,1,0),(0,0,1); their images are (1,0),(0,1). Thus $3 = 1 + 2$, and the proof visibly constructs the image basis.

A. Boundary and hypothesis audit

The numerical formula uses finite dimensions. With an infinite-dimensional domain one needs cardinal-dimension statements and appropriate basis principles; subtracting "infinite rank" from "infinite dimension" is not valid arithmetic. Sense keeps this studio's finite-dimensional input contract.

VI. ILLUSTRATIVE FAILURE AND REPAIR

The following is an authored diagnostic scenario. Changed assumptions define a new contract; they do not retroactively prove the original statement.

A. Failure and correction

This is an authored teaching trace, not a recorded agent run.

1. **Ego → Memory:** register the §1 theorem, imports, and requested exports before beginning the proof.
2. **Intellect → Action:** choose the bridges in §3 and carry out the numbered activities in §4; transmit only established E-exports along the §2 DAG.
3. **Sense → responsible node:** test the specific boundary/failure in §5 and the node's individual audit. A failing candidate is withheld; downstream nodes may not use it.
4. **Memory + Intellect:** identify whether the error is an omitted condition, an invalid inference, or an attempted extension beyond the theorem. Restore the exact contract or explicitly state a different one.
5. **Action:** redo the affected mathematical activity using the repaired condition or inference. Recheck consuming nodes, rather than feeding their conclusions back as premises.
6. **Ego:** publish only the supported export with its assumptions and proof location. A hypothetical revised export is v2; the written proof here is the v1 exposition, not a claim that this repair was executed.

B. Hypothesis boundary

The numerical formula uses finite dimensions. With an infinite-dimensional domain one needs cardinal-dimension statements and appropriate basis principles; subtracting "infinite rank" from "infinite dimension" is not valid arithmetic. Sense keeps this studio's finite-dimensional input contract.

C. Version and recheck

Treat any changed assumption as a new contract. Recheck the revised export and all of its descendants. A backward diagnostic never establishes an earlier theorem. This is an authored teaching trace, not an execution log.

Provenance. This authored proof is complete relative to its declared imports. Reuse each export with its assumptions, proof, and version. The original Markdown remains available for comparison.